



Universidad de Jaén

Tema 2.4 Jakarta EE

Introducción a la capa de servicios Patrón DAO, Validación del modelo, JPA

José Ramón Balsas Almagro
Departamento de Informática
Universidad de Jaén
jrbalsas@ujaen.es

Este obra está bajo una [Licencia Creative Commons](https://creativecommons.org/licenses/by/4.0/)
Atribución-CompartirIgual 4.0 Internacional.

v 3.2.1

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



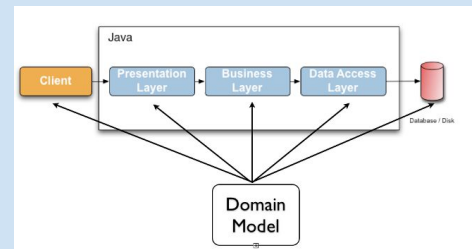
Objetivos

- Conocer las características del **patrón de diseño DAO** para acceso básico al modelo de datos de una aplicación web
- Conocer y saber utilizar el mecanismo de **Bean Validation** para la validación de entidades en el modelo de aplicaciones JEE
- Conocer aspectos básicos sobre **conexión a bases de datos** en servidores JEE y la importancia de utilizar **pool de conexiones** en aplicaciones web
- Conocer la **arquitectura y conceptos básicos de JPA** como sistema de persistencia de objetos sobre bases de datos relacionales, **ORM**, en JEE
- Saber **configurar JPA** desde una aplicación JEE
- Conocer y saber utilizar las **operaciones básicas de persistencia de datos** con JPA



Conexión con el Modelo de Datos en una aplicación web

- **Arquitectura en una aplicación empresarial JEE**
 - Separación de competencias



Fuente: [Hibernate Validator Reference](http://hibernate.org/validator/reference/)

- La **capa de presentación** (aplicación web) debe encargarse únicamente de **aspectos de control de la interacción con el usuario y presentación** de los contenidos web a través de HTTP: *controlador y vistas*
 - La **capa de acceso a datos (persistencia)** debe ser transparente a la aplicación web
 - El **modelo de dominio** y la **validación de datos** son comunes a todas las capas

3

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Data Access Object, DAO Pattern

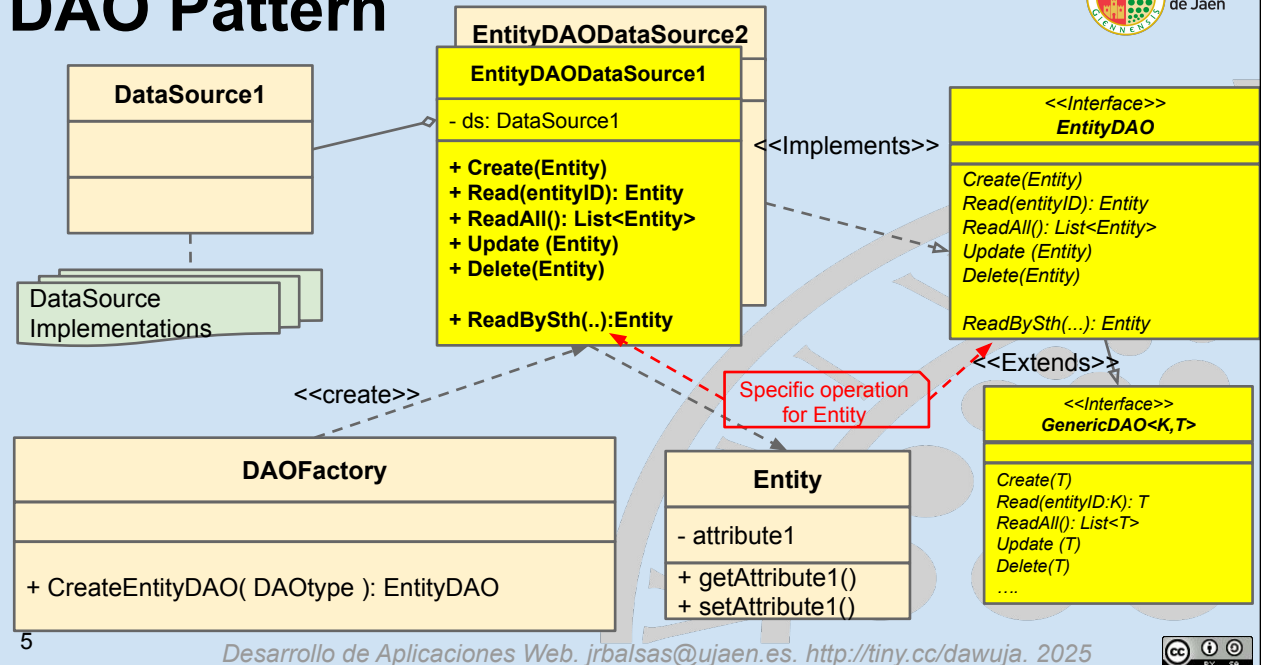
- **Objetivo:** ocultar los aspectos de acceso al sistema de almacenamiento de información
- **Elementos:**
 - **Entidades** del modelo de dominio que se van a gestionar
 - **Interfaz de acceso** a los datos: operaciones habituales para acceder a los datos, CRUD (Create, Read, Update, Delete), con independencia del sistema de almacenamiento utilizado
 - En ocasiones puede existir una **interfaz genérica** para homogeneizar todos los DAO con operaciones básicas
 - **Implementaciones específicas de la interfaz de acceso a los datos, DAO**, dependiendo de los mecanismos de almacenamiento. E.g. *Fichero, Base de datos relacional, base de datos NoSQL, Servicio REST, etc.*
 - **Mecanismos de selección** para obtener instancias concretas de objetos DAO dependiendo del sistema de conexión y almacenamiento requerido, e.g. **Factoría de DAOs, Qualifiers**

4

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



DAO Pattern



5

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Características de un DAO

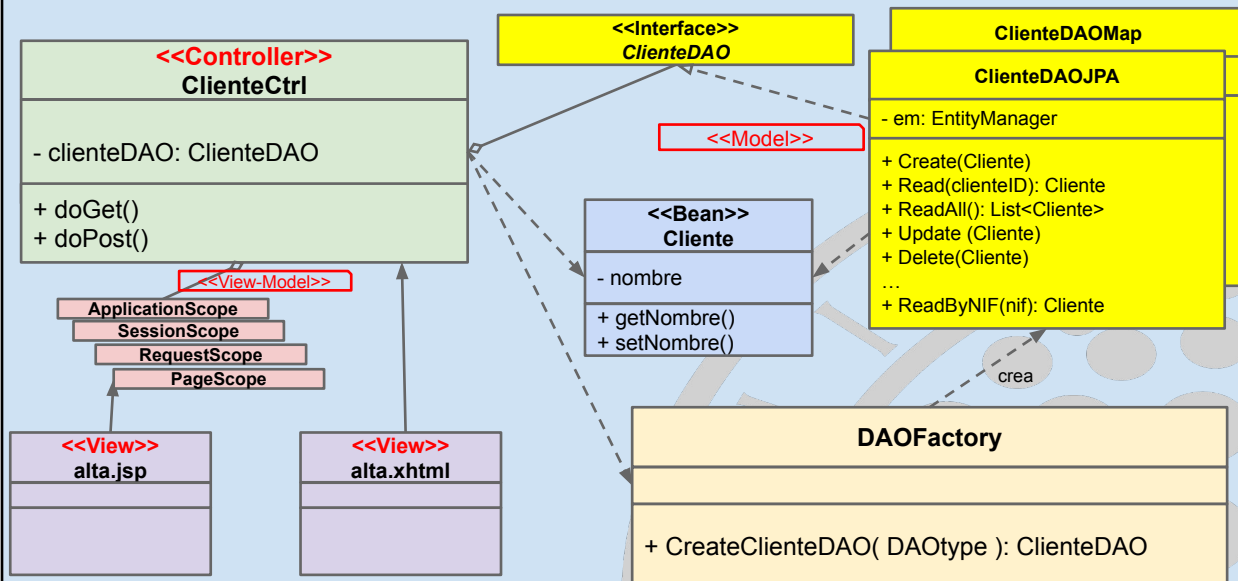
- Un front-end web accederá a los datos del modelo a través de Servicios o DAOs específicos (*Repositories*)
- **Funciones**
 - **Gestión del canal de comunicación** con el sistema de almacenamiento o persistencia. E.g. *solicitar/liberar conexión con la Base de Datos*
 - **Ocultar detalles de interacción** con el sistema de almacenamiento. E.g. *construir consultas SQL para acceder a la información*
 - **Transformar (Mapping)** los elementos del sistema de almacenamiento en entidades del modelo. E.g. *instanciar un objeto del modelo a partir de un registro de una tabla*
- **Consideraciones de uso**
 - Normalmente un DAO por entidad principal
 - Usados por controlador o servicio cuando se necesita **acceso CRUD a entidades**
 - Las entidades pueden estar encapsuladas total o parcialmente en **DTOs (Data Transfer Objects)** cuando deban adaptarse durante su manipulación, E.g. *ClienteForm (DTO) → Cliente (Entidad del modelo de dominio)*
 - Para operaciones que implican **lógica de negocio** es mejor encapsular los DAOs implicados en **capas adicionales de Servicio** (capa de servicio, se verá en 4º en Aplicaciones Empresariales)

6

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Arquitectura MVC aplicación web



7

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo de implementaciones DAO sobre un Map o JPA

Interfaz genérica DAO

```
package daw.club.model;
import java.util.List;
```

```
public interface GenericDAO<K,T> {

    public T buscard(K id);
    public List<T> buscaTodos();
    public boolean crea(T c);
    public boolean guarda(T c);
    public boolean borra(K id);
}

public interface ClienteDAO
    extends GenericDAO<Integer,Cliente> {
    //specific DAO methods for entity Cliente
    public Cliente buscaNIF(String nif);
}
```

Inicialización DAO (en Controlador)

```
@Inject @DAOJpa //Qualifier para indicar la implementación
private ClienteDAO clienteDAO;
```

Posibles Implementaciones DAO

```
@DAOMap //Qualifier creado por desarrollador
@ApplicationScoped
public class ClienteDAOMap implements ClienteDAO {
    //Implementation of interface methods
    public Cliente buscard(Integer id) { ... };
    public List<Cliente> buscaTodos() { ... };
    public boolean crea(T c) { ... };
    public boolean guarda(T c) { ... };
    public boolean borra(Integer id) { ... };
    public Cliente buscaNIF(String nif) { ... };
    ...
}

@DAOJpa //Qualifier creado por desarrollador
@RequestScoped
public class ClienteDAOJpa implements ClienteDAO { ... }
```

```
//Definición de Qualifier. File: DAOJpa.java
@Qualifier
@Retention(RUNTIME)
@Target({METHOD, FIELD, PARAMETER, TYPE}) //valid uses
public @interface DAOJpa { }
```

8

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo DAO para pruebas

@DAOMap @ApplicationScoped

public class ClienteDAOMap implements ClienteDAO{

private Map<Integer, Cliente> clientes=null;

private int idCliente = 1; //last clientID

public ClienteDAO() {

clientes = new HashMasp<>();

}

@Override

public List<Cliente> buscaTodos() {

return clientes.values().stream().collect(Collectors.toList());

}

@Override

public Cliente buscald(Integer id) {

return new Cliente (clientes.get(id));

}

@Override

public boolean crea(Cliente c) {

c.setld(idCliente++);

clientes.put(c.getld() , new Cliente(c));

return true;

}

- Simula una colección de datos, almacenados en memoria mediante un HashMap y compartidos por todos los usuarios

@Override

public boolean guarda(Cliente c) {

boolean result=false;

if (clientes.containsKey(c.getld())) {

Cliente nc=new Cliente(c);

clientes.replace(nc.getld(), nc);

result=true;

return result;

}

@Override

public boolean borra(Integer id) {

boolean result=false;

if (clientes.containsKey(id)) {

clientes.remove(id);

result = true;

return result;

}

9

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Métodos de acceso a sistemas de almacenamiento o persistencia

- Atendiendo a la forma de acceder al recurso

- **Sistema de archivos:** serialización, csv, ...

- **Servicios de Directorio:** LDAP, Active Directory

- Sistemas de acceso a **Bases de Datos**

- Relacionales, basados en SQL: *JDBC* (JSR-221), *Spring JDBCTemplate*, *JDBI*, *JOOQ*, ...

- NoSQL: *MongoDB*, *Cassandra*, *BigTable*, *Jakarta NoSQL*

- **Sistemas de Mapeado de Objetos:**

- Sobre BD relacionales, ORM (Object-Relational Mapping): *MyBatis*, *Hibernate*, *Jakarta Persistence*, *JPA* (3.1)

- Sobre BD NoSql: *Jakarta NoSQL* (borrador)

- Sobre cualquier recurso: *JDO* (JSR-243), *Jakarta Data* (1 → Jakarta 11)

- **Servicios Web:**

- Basados en SOAP: *Jakarta XML Web Services*, *JAX-WS* (3.0)

- Basados en REST: *Jakarta Restful Web-Services*, *JAX-RS* (3.1)

- ...

10

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025

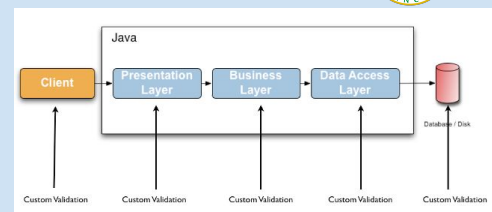


Validación de datos en el Modelo

Bean Validation

Validación de Datos

- Tarea común en todas las capas de la aplicación, costosa y propensa a errores
- **Solución:** unificar como parte del modelo de datos
- **Bean Validation API 3.0**
 - Declaración de restricciones y metadatos de validación en JEE
 - Uso de **anotaciones** para indicar:
 - **Restricciones** en atributos o incluso parámetros: predefinidas o personalizadas
 - Definición de **mensajes de error personalizados**
 - Un **validador** se encarga de verificar que se cumplen en tiempo de ejecución
 - Integrado en JEE >=6, e.g. Glassfish
 - Usar biblioteca *Hibernate-validator* en Tomcat
 - (artifactID org.hibernate.hibernate-validator en proyecto Maven)



Fuente: [Hibernate Validator Reference](#)

Ejemplo de anotaciones de validación

```
public class Cliente {  
    private Integer id;  
    @Size(min=5,max=50)  
    private String nombre;  
  
    @NotEmpty  
    private String apellidos  
    @Pattern(regexp="\\d{8}-?[a-zA-Z]*")  
    private String dni;  
  
    @Min(18)  
    @Max(119)  
    private int edad;  
    private boolean socio;  
  
    @Future  
    private LocalDate fechaRenovacion;  
    //...  
}
```

// dni 8 dígitos, guión opcional y letra

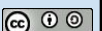
13

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025

Anotaciones de validación predeterminadas para Beans

Anotación	Descripción
<code>@AssertFalse/True</code>	Una propiedad booleana es verdadera/falsa
<code>@DecimalMax/Min(value=, inclusive=)</code>	Valor Máximo de una propiedad decimal. <i>inclusive=true</i> para $\leq$
<code>@Digits(integer=,fraction=)</code>	Número exacto de dígitos enteros y/o decimales
<code>@Future, @Past, @Future/PastOrPresent</code>	Valor de fecha futura o pasada respecto a la actual. Actual o futura y Actual o pasada
<code>@Max/Min(value=), @Positive/Negative, @Positive/NegativeOrZero</code>	Valor entero menor/mayor o igual
<code>@NotNull, @Null, @NotEmpty</code>	La propiedad no debe/debe ser igual a null. <i>Empty</i> para cadenas.
<code>@Pattern(regexp=,flag=), @Email</code>	La propiedad satisface una expresión regular según un criterio de comparación (flag)
<code>@Size(min=,max=)</code>	Longitud mínima o máxima de una cadena, array, colección o map
<code>@Valid</code>	Validación recursiva para tipos no primitivos. e.g. atributo de tipo bean

14

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025

Validadores personalizados

```
@Constraint(validatedBy = DniValidator.class)
@Retention(RUNTIME)
@Target(ElementType.FIELD)
public @interface Dni { //Custom validator annotation

    //Default annotation attribute value
    String message() default "DNI incorrecto";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}
```

Utilización

```
public class Cliente {

    private Integer id;

    @Size(min = 4, max = 25)
    private String nombre;

    //@Pattern(regexp = "\\d{8}-?[a-zA-Z]" )
    @Dni //Apply custom validator
    private String dni;

    private boolean socio;
    //...
}
```

```
/** Sample Spanish DNI Custom validator*/
public class DniValidator
    implements ConstraintValidator<Dni, String> {

    @Override
    public boolean isValid(String dni,
        ConstraintValidatorContext context) {

        String regexDni="(\\d{8})-?([a-zA-Z])";
        Matcher matcher = Pattern.compile(regexDni).matcher(dni);
        if (!matcher.matches()) {
            //Dni format not valid
            return false; //default format message
        }

        String numeroDni = matcher.group(1);
        String letraDni = matcher.group(2);

        // Calculate DNI checksum letter
        int resto = Integer.parseInt(numeroDni) % 23;
        String letra = "TRWAGMYFPDXBNJZSQVHLCKE";

        if (letra.charAt(resto) != letraDni.charAt(0) ) {
            // DNI verification error
            //Custom error validation message
            context.disableDefaultConstraintViolation();
            context.buildConstraintViolationWithTemplate(
                "El dni %s no es válido".formatted(dni))
                .addConstraintViolation();
        }

        return true; //valid DNI
    }
}
```

15

 Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025


Mensajes de error de validación

- Bean Validation dispone **mensajes de error predeterminados** y adaptados al idioma (*locale*)

Ejemplo de error para violación de `@Pattern(regexp="\\d{8}-?[a-zA-Z]" )`

Mensaje de error predeterminado → *Tiene que corresponder a la expresión regular "d{8}-?[a-zA-Z]"*

- **Mensajes de error personalizados:**

- **Atributo message=""** en restricción del bean
 - `@Min(value=18, message="Sólo se admiten mayores de edad")`
 - **Admite EL** para acceder a datos de validación:
 - `@Min(value=18, message="La edad ${validatedValue} no supera el mínimo permitido: {value}")`
- Entradas fichero **ValidationMessages.properties** (src/main/resources):
 - `@Min(value=18, message="{cliente.edad.min}")`
 - `cliente.edad.min="Sólo se admiten mayores de edad ({value})"`

- **Los Facelets muestran errores en etiquetas `<h:message>`**

16

 Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025


Acceso a BBDD relacionales en JEE

DataSources

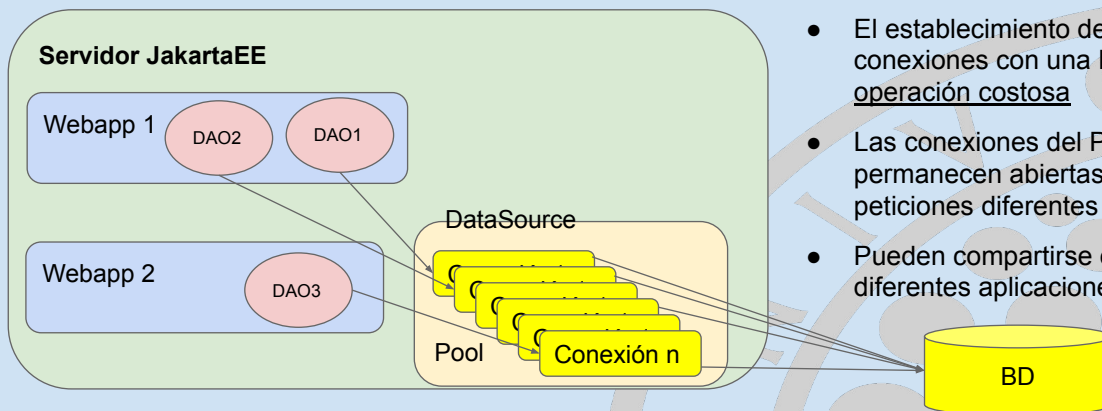
Acceso a bases de datos en JEE

Conceptos relacionados:

- **Recursos (*Resources*)**, objetos que permiten la conexión a BDs, sistemas de mensajería, etc.
Pueden ser creados:
 - Desde el servidor de aplicaciones: interfaz web, CLI, API REST servidor
 - Desde la propia aplicación: web.xml, código, ...
 - Se pueden obtener mediante DI con la anotación `@Resource` (`..id jndi ...`)
- **JNDI (*Java Naming and Directory Interface*)**, identificador único de recurso para localizarlo en un servidor de aplicaciones JEE, e.g.
 - **Local** a la aplicación `java:app/jdbc/gestClub`
 - **Global** a todas las aplicaciones `java:global/jdbc/gestClub` (utilizar cuando es creado por el servidor)
- **DataSource**, recurso JDBC que permite el acceso a una base de datos relacional.
 - **Driver** que se utilizará
 - **Datos de configuración** para acceso a BD: host, usuario, clave, bd...
 - **Nombre JNDI** para localizarlo
 - La aplicación los utiliza para **obtener conexiones** a la BD
 - Optimizan el rendimiento de la BD reutilizando conexiones mediante **Pool de conexiones**

Pool de conexiones a BD

- **Motivación:** ¿cómo optimizar el acceso a la base de datos?



- El establecimiento de conexiones con una BD es una operación costosa
- Las conexiones del Pool permanecen abiertas entre peticiones diferentes
- Pueden compartirse entre diferentes aplicaciones

19

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Definición de conexiones a bases de datos en JEE

- Es necesario el **driver de acceso a la BD**:
 - Proporcionado por el servidor, e.g. Payara dispone de drivers para *Derby* y *H2Database*
 - Proporcionado por la aplicación, e.g. dependencia maven

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <version>2.1.214</version>
  <scope>runtime</scope> //En payara solo es necesario ámbito provided
</dependency>
```

- La aplicación define la conexión mediante una **cadena de conexión**, e.g.

Driver	Cadena conexión
com.mysql.jdbc.Driver	jdbc:mysql://hostname:port/databaseName
oracle.jdbc.driver.OracleDriver	jdbc:oracle:thin:@hostname:port:databaseName
org.h2.jdbcx.JdbcDataSource	jdbc:h2:~/h2db/dawclub;MODE=LEGACY;AUTO_SERVER=TRUE (también rutas absolutas: /tmp/... o c:/...) jdbc:h2:mem:dawclub;MODE=LEGACY;DB_CLOSE_DELAY=-1
org.apache.derby.jdbc.ClientDriver	jdbc:derby://host:port/databaseName
org.apache.derby.jdbc.EmbeddedDriver	jdbc:derby:path/databaseName

20

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Definición de DataSources

- Definición genérica declarativa en JEE

- Descriptor de despliegue, web.xml

```
<data-source>
<name>java:global/jdbc/gestClub</name>
<class-name>org.h2.jdbcx.JdbcDataSource</class-name>
<url>jdbc:h2:~/h2db/dawclub;MODE=LEGACY;AUTO_SERVER=TRUE</url>
<user>dawuser</user>
<password>dawuser</password>
<initial-pool-size>2</initial-pool-size>
<max-pool-size>10</max-pool-size>
<min-pool-size>5</min-pool-size>
</data-source>
```

<!-- JNDI resource -->
<!-- Driver -->
<!-- Create DB in user home folder -->
<!-- Data pool config -->

- Anotación `@DataSourceDefinition` [+info](#)

- Específica del Servidor

- Desde la propia aplicación (fichero de configuración específico del servidor)

- Tomcat: fichero META-INF/context.xml
 - Glassfish/Payara: fichero glassfish-resources.xml

- Desde el propio servidor

- Glassfish/Payara: Consola web o app en línea de comandos (asadmin)
 - Tomcat: fichero server.xml en servidor

- Definición programática con CDI

```
@Produces
DataSource createDataSource () { /*create and return DataSource for DI */ }
```

21

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



JPA

Jakarta Persistence API

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Jakarta Persistence API, JPA

- Estándar JEE para persistencia de objetos, v3.1
- Facilita la transformación (mapeado) de objetos hacia o desde tablas en bases de datos relacionales: **Object Relational Mapping (ORM)**
- **Ventajas**
 - API de acceso a BD relacionales de más alto nivel que JDBC
 - Simplificación y homogeneización de operaciones CRUD habituales
 - Diferentes implementaciones: *EclipseLink* (disponible en Payara), *Hibernate*, *DataNucleus*...
- **Posibles inconvenientes**
 - Mayor curva de aprendizaje
 - Un mal uso puede dar lugar a operaciones poco eficientes sobre la BD
 - No soporta de forma directa funcionalidades específicas de BD, e.g. triggers, procedimientos almacenados, vistas, etc.

23

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025

Características JPA

- Basado en **anotaciones** sobre clases
- **Soporta Bean Validation**
 - no permite almacenar información que no verifique restricciones definidas
- **Otras características:**

(se estudiarán en Aplicaciones Empresariales, 4º curso)

 - Soporte de **transacciones**
 - Soporta **relaciones entre entidades**: *one-to-one*, *one-to-many*, *many-to-one*, *many-to-many*

24

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025

Entidades JPA

• Objetos gestionados por el ORM

- Clases anotadas con `@Entity`
- `@Id`, atributo/s considerado llave primaria
 - con `@GeneratedValue` para indicar valores automáticos, e.g. auto-incrementales
- Por lo general están vinculadas a una tabla de la BD
- Cada atributo (no final) se mapea a una campo de una tabla

• Otras anotaciones (opcionales)

- `@Table` nombre de la tabla sobre la que se almacenará el objeto (por defecto el nombre de la clase)
- `@Column("campo")` campo de la tabla donde se almacenará un atributo o propiedad (por defecto el mismo nombre)
- `@Transient` atributo que no se almacenará en la tabla

25

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo de mapeado de objetos con JPA

```
@Entity
@Table(name="CLIENTES")
public class Cliente implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) //atributo llave primaria //autoincremental
    private Integer id;

    @Column(name="nom_ape", length = 100)
    @Size(min = 4, max = 100,
        message = "La longitud ${validatedValue} debe estar entre {min}
            y {max} caracteres")
    private String nombre;

    @Pattern(regexp="\\d{8}-?[a-zA-Z]",
        message="{cliente.dni.formato}")
    private String dni;

    private boolean socio;

    @transient
    private int minutos_conectado; //atributo no persistente
}
```

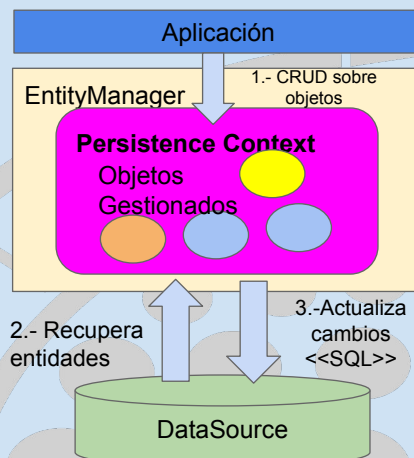
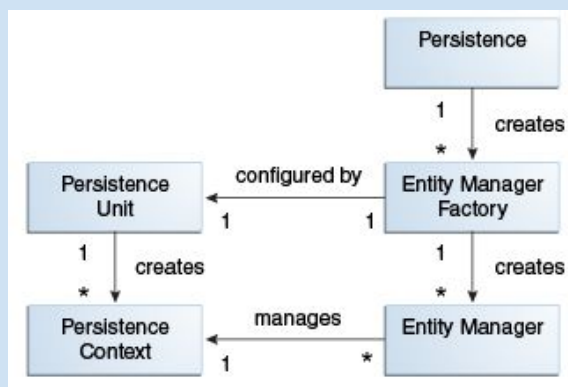
26

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Arquitectura JPA

Fuente: EclipseLink



27

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Conceptos JPA en aplicaciones web

- **Persistence Unit, PU:** Definición de las entidades que serán gestionadas (puede haber varias) y su ubicación, e.g. BBDD relacional
 - Configuración en [/META-INF/persistence.xml](#) (carpeta src/main/resources en proyecto maven)
 - Vinculadas a un **DataSource**, e.g. definido con JNDI (en web.xml)
- **EntityManager, EM**
 - Realiza **operaciones sobre entidades** gestionadas en **contexto de persistencia** (*Persistence Context, PC*), e.g. *find*, *remove*, *query*, etc.
 - Un contexto de persistencia está vinculado a una PU
- **Transacciones**
 - Las operaciones realizadas sobre entidades de un PC son agrupadas en transacciones. Solo se reflejan en la BD si finaliza la transacción (*commit*)

Los contenedores JEE se encargan de:

- Proporcionar un **EntityManager** mediante DI


```
@PersistenceContext (unitName="clubPU") //unitName solo necesario si hay varias
EntityManager em;
```
- Gestionar automáticamente las transacciones en clases CDI o en sus métodos anotados con **@Transactional**. Necesario en operaciones que impliquen modificaciones

28

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Configuración de una Unidad de Persistencia

resources/META-INF/persistence.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="3.0" xmlns="http://jakarta.ee/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://jakarta.ee/xml/ns/persistence
        http://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd">

    <persistence-unit name="ClubPU" transaction-type="JTA">
        <!-- Optional: select JPA provider implementation, e.g EclipseLink -->
        <!-- <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider> -->
        <!-- Use JNDI Datasource (e.g. JEE connection pool in web.xml) -->
        <jta-data-source>java:global/jdbc/gestClub</jta-data-source>
        <properties>
            ...
            <!-- EclipseLink specific properties e.g. debug level -->
            <property name="eclipselink.logging.level" value="INFO"/>
            <property name="eclipselink.logging.parameters" value="true"/>
            <property name="eclipselink.logging.level.sql" value="FINE"/>
        </properties>
    </persistence-unit>
</persistence>
```

Transacciones
gestionadas por
servidor

Permite la
visualización de
las instrucciones
SQL usadas por
JPA

29

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Creación esquema y precarga de datos

- Para una PU puede indicarse **cómo debe inicializarse el esquema** de la BD
 - Opciones:** **none** (defecto), **create**, **drop-and-create**, ...
 - Tipos de creación:**
 - metadata**, a partir de entidades (por defecto)
 - script**, script con instrucciones DDL específicas
 - metadata-then-script**, ambas opciones
 - Se puede proporcionar un **script con instrucciones SQL** para precargar datos en las tablas

```
<persistence version="3.0" xmlns="http://jakarta.ee/xml/ns/persistence" ...>
    <persistence-unit name="ClubPU" transaction-type="JTA">
        ...
        <properties>
            <!-- JPA table creation mode, e.g. none, create, drop-and-create -->
            <property name="jakarta.persistence.schema-generation.database.action" value="drop-and-create"/>
            <!-- Populate with sample data for managed entities on starting app -->
            <property name="jakarta.persistence.sql-load-script-source" value="/META-INF/sql/sampledata.sql"/>
            <!-- Specific db operations, e.g. create tables not managed by JPA -->
            <property name="jakarta.persistence.schema-generation.create-source" value="metadata-then-script"/>
            <property name="jakarta.persistence.schema-generation.create-script-source" value="/META-INF/sql/schema.sql"/>
        </properties>
    </persistence-unit>
</persistence>
```

30

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Operaciones básicas *EntityManager*

- Permiten que un EM recopile, durante una transacción, operaciones sobre entidades gestionadas que serán aplicadas de forma permanente si esta finaliza adecuadamente
- **Tipos:**
 - **find (Entidad.class, id)**, localizar una entidad por su llave primaria (i.e. SELECT)
 - **persist (objeto)**, convierte un objeto en entidad añadiéndolo al contexto de persistencia y almacenándolo (i.e. INSERT)
 - **remove (entidad)**, elimina una entidad del contexto de persistencia. (i.e. DELETE) OJO, debe estar gestionada por éste, e.g. usando find, o lanza una excepción *IllegalArgumentException*
 - **merge(objeto)**, localiza una entidad con el id del objeto indicado y la actualiza con su información (i.e. UPDATE). Si no la encuentra, crea una nueva (i.e. INSERT). Devuelve la entidad gestionada.
 - **createQuery**, consulta de entidades usando
 - *Jakarta Persistence Query Language (JPQL)*, algunas semejanzas con SQL
 - Criterio API, uso de sintaxis de Java para construcción de consultas
 - **createNativeQuery**, consultas nativas con SQL propio de la BD

31

DAO JPA con operaciones básicas

```
@RequestScoped //Eligible for Dependency Injection
@DAOJpa
@Transactional //Transactions managed by server
public class ClienteDAOJPA
    implements ClienteDAO, Serializable {

    @PersistenceContext(unitName = "ClubPU")
    private EntityManager em;

    @Override
    public Cliente buscald(Integer id) {
        return em.find( Cliente.class, id);
    }

    @Override
    public boolean crea(Cliente c) {
        boolean creado = false;
        try {
            em.persist( c );
            creado = true;
        } catch (Exception ex) {
            logger.log(Level.SEVERE, ex.getMessage(), ex);
        }
        return creado;
    }
}
```

```
@Override
public boolean guarda(Cliente c) {
    boolean guardado = false;
    try {
        c = em.merge( c );
        guardado = true;
    } catch (Exception ex) {
        logger.log(Level.SEVERE, ex.getMessage(), ex);
    }
    return guardado;
}

@Override
public boolean borra(Integer id) {
    boolean borrado = false;
    try {
        Cliente c = null;
        c = em.find(Cliente.class, id);
        em.remove(c);
        borrado = true;
    } catch (Exception ex) {
        logger.log(Level.SEVERE, ex.getMessage(), ex);
    }
    return borrado;
}
```

32

Consultas JPA

```
@Override
public List<Cliente> buscaTodos() {
    List<Cliente> lc = null;
    try {
        Query q = em.createQuery("Select c from Cliente c"
                                , Cliente.class);
        lc = q.getResultList(); //get múltiple entities
    } catch (Exception ex) {
        logger.log(Level.SEVERE, ex.getMessage(), ex);
    }
    return lc;
}
```

- Pueden representarse como metadatos

```
@NamedQuery(
    name="findCustomerWithDNI",
    query="Select c from Cliente c where c.dni=:dni"
)
@Entity
class Cliente { ... }

public Cliente buscaByNIF(String dni) {
    Cliente c = null;
    try {
        TypedQuery<Cliente> q = em.createQuery("findCustomerWithDNI", Cliente.class);
        q.setParameter("dni", dni); //set query parameter value
        c = q.getSingleResult(); //get single entity
    } catch (Exception ex) { ... }
    return c;
}
```

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Consultas parametrizadas

- Se utilizan cuando una instrucción SQL se realiza repetidas veces con valores (no campos) diferentes (parametrizadas).
- Facilidad para construir la instrucción SQL con los valores. Admiten **parámetros ordinales**, **?**, o con nombre, e.g. **:dni**
- Ejecución más rápida
- +info [JPQL en Jakarta EE Tutorial](#)

Seguridad en consultas SQL

Ataques de inyección SQL

- En general, hay que filtrar cualquier dato que se recibe del cliente para prevenir valores no deseados
- ... especialmente si van a formar parte de una consulta SQL, ya que podrían "confundir" al analizador SQL de la base de datos consiguiendo inyectar operaciones adicionales no deseadas:

Ejemplo inseguro ✗

```
String qry = "select c from Clientes c where c.dni=" + request.getParameter("dni") + " ";
→ select c from Clientes c where c.password=' 1'; drop table Clientes;--'
```



- Las consultas parametrizadas garantizan que los parámetros NO serán procesados por el analizador SQL y solo se utilizan como datos

Ejemplo seguro: ✓

```
String plantillaSQL = "Select c from Cliente c where c.dni=:dni";
TypedQuery<Cliente> q=em.createQuery( plantillaSQL, Cliente.class )
    .setParameter("dni", request.getParameter("dni") );
List<Cliente> lc = q.getResultList();
```



Mapeado manual de objetos

- **Motivación**

- Consultas con registros que quieren mapearse en **objetos no gestionados** por JPA

- **Opciones**

- Utilizar **sintaxis de constructores de la clase** destino con JPQL
 - Debe utilizarse el **nombre completo de la clase**
 - La clase debe disponer del **constructor parametrizado** adecuado

```
string qry="Select new com.daw.clubjsf.DatosSocioDTO( c.dni, c.nombre ) "
      +"from Cliente c where socio=true";
List<DatosSocioDTO> lc = em.createQuery( qry, DatosSocioDTO.class)
      ).getResultList();

// clase temporal, e.g. para pasar datos a un formulario
class DatosSocioDTO {
    private String dni;
    private String nombre;
    DatosSocioDTO ( String dni, String nom ) {
        this.dni=dni;
        this.nombre=nom;
    }
    //get/set
}
```

- Uso de **consultas nativas**

35

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Consultas Nativas

- **Utilidad**

- Uso de **consultas SQL complejas o funcionalidades específicas** de la BD no soportadas por JPQL
- Operaciones en **tablas no gestionadas** por JPA, e.g. *insert*, *update*, *delete*...
- **Consultas que no recuperan entidades**, e.g. DTOs, campos individuales, campos calculados, ...

- **Uso básico**

- `em.createNativeQuery ( consulta )`

- **Opciones para recuperación de datos:**

- Recuperación y conversión manual de campos
- Definición de un mapeado de campos

36

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Consultas nativas: recuperación manual de campos

- Cada registro de la consulta se devuelve como:
 - *Object*, si el registro solo tiene un campo
 - *Object[]*, vector de objetos, si tiene varios campos.
 - Se accede a cada campo por posición
 - es necesaria conversión de tipos

Ejemplo:

```
class FacturacionClienteDTO {
    private Integer idCliente;
    private String nombre;
    private float totalFacturado;

    FacturacionClienteDTO (Integer id, String nombre
        , Float facturacion) {...}

    //get/set
}

public List<FacturacionClienteDTO> recuperaFacturacionCliente() {
    List<FacturacionClienteDTO> l = new ArrayList<>();
    try {
        string qry= "SELECT c.id, c.nombre, sum(f.importe) FROM Cliente c JOIN Facturas f ON c.id=f.idCliente";
        Query q = em.createNativeQuery( qry );
        List<Object[]> lobjs = q.getResultList();
        for (Object[] row : lobjs) {
            l.add( new FacturacionClienteDTO( (Integer) row[0], row[1].toString(), (Float) row[2] ) );
        }
    } catch (Exception ex) {
        logger.log(Level.SEVERE, ex.getMessage(), ex);
    }
    return l;
}
```

37

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Consultas nativas: mapeado manual de campos

- **@SqlResultSetMapping**, utilizado en una entidad, permite definir mapeados manuales de campos en objetos no gestionados:

```
@Entity
@SqlResultSetMapping(
    name = "DatosFacturacionDTOMapping",
    classes = @ConstructorResult(
        targetClass = DatosFacturacionDTO.class,
        columns = {
            @ColumnResult(name = "id", type = Integer.class),
            @ColumnResult(name = "name"),
            @ColumnResult(name = "total", type= Float.class)
        }
    )
)
public class Cliente {
    ...
};

List<DatosFacturacionDTO> recuperaFacturacionClientes() {
    List<FacturacionClienteDTO> lfc = null;
    try {
        string qry= "SELECT c.id as id, c.nombre as name, sum(f.importe) as total "
            +" FROM Cliente c JOIN Facturas f ON c.id=f.idCliente";
        Query q = em.createNativeQuery( qry,
            "DatosFacturacionDTOMapping");
        lfc = q.getResultList();
    } catch (Exception ex) {
        logger.log(Level.SEVERE, ex.getMessage(), ex);
    }
    return lfc;
}
```

38

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Conclusiones

- En una aplicación web, el **patrón de diseño DAO** permite abstraer los detalles de acceso al sistema de persistencia de la información
- La **validación de datos del modelo de dominio** debe realizarse en todas las capas de una aplicación y, por lo tanto, debe **centralizarse** para simplificar el proceso y evitar errores
- La especificación **Bean Validation** permite la definición de restricciones de datos en objetos y la validación de los mismos
- La gestión de las **conexiones a una BD en una aplicación web** es crítica y debe **delegarse al servidor de aplicaciones** para optimizar su uso
- **JPA** es un módulo ORM para simplificar la persistencia de objetos en bases de datos relacionales en Java.

39

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025

Bibliografía

- Introduction to Bean Validation, en *Jakarta EE Tutorial, Web Profile*. Eclipse Foundation, 2024. Disponible en <https://jakarta.ee/learn#documentation>
- Introduction to Persistence, en *Jakarta EE Tutorial, Web Profile*. Eclipse Foundation, 2024. Disponible en <https://jakarta.ee/learn#documentation>
- D. R. Heffelfinger, *Jakarta EE Application Development: Build Enterprise Applications with Jakarta CDI, RESTful Web Services, Persistence, and Security*, Second edition. Packt Publishing, 2017. [[Recurso Electrónico](#)]

Lecturas Complementarias

- *Jakarta Persistence API*. Eclipse Foundation, 2021. Disponible en <https://jakarta.ee/specifications/persistence/3.1/>
- *Hibernate Validator-JSR 380 Reference Implementation: Reference Guide*. JBoss, 2021. Disponible en: https://docs.jboss.org/hibernate/stable/validator/reference/en-US/html_single/

40

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025